A Data Science Bootcamp

Winter in Data Science (WiDS) 5.0

Name:

Umesh Motiwale
(24b1240)

EndTerm Project Report of

LiDAR Cone Detection (LCD)

Under the mentorship of
Kasak & Bhoomi

Indian Institute of Technology, Bombay

Academic Year: 2025–26

Contents

1 Machine Learning **3**
1.1 Types of Machine Learning . . . 3

2 Supervised Learning **3**
2.1 Types of Supervised Learning . . . 3
2.1.1 Classification . . . 3
2.1.2 Regression . . . 3

3 Unsupervised Learning **3**
3.1 Types of Unsupervised Learning . . . 4
3.1.1 Clustering . . . 4
3.1.2 Association . . . 4

4 Reinforcement Learning **4**

5 Algorithms **4**

6 Types of Clustering **5**
6.1 Clustering Using DBSCAN . . . 5

7 Foundation of Linux Administration **5**
7.1 Introduction to Linux and Shells . . . 5
7.2 Navigation and File Management . . . 5
7.3 File Manipulation and Search . . . 6
7.4 Permissions and Ownership . . . 6
7.5 Processes and Environment . . . 6

8 Introduction to ROS 2 Fundamentals and Development Workflow **7**
8.1 ROS 2 CLI Tools . . . 7
8.2 Client Libraries and Build Tools . . . 7
8.3 ROS 2 Workspace Structure . . . 7
8.4 Development Workflow: Creating a Node . . . 8
8.5 Code Logic: Publisher/Subscriber Pattern . . . 8
8.6 Publisher and Subscriber Communication Model . . . 8

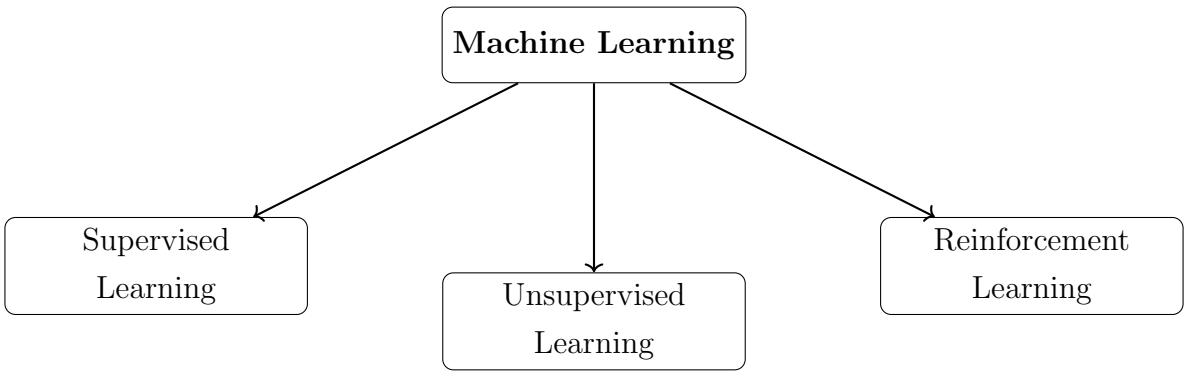
9 LiDAR-only Cone Detection Using ROS 2 **9**
9.1 Problem Statement . . . 9
9.2 Sensor Used: LiDAR . . . 9
9.3 Point Cloud Representation . . . 9

9.4 Cone Detection Pipeline 9
9.5 Use of Rosbag 10
9.6 Final Output and Results 10
9.7 Visualization using rViz 12
9.8 Learning Outcomes 12

1 Machine Learning

Machine learning is a technique in which computers learn from past data to make predictions or decisions without explicit programming.Instead of writing fixed rules, we give the machine data + algorithms, and it finds patterns on its own.

1.1 Types of Machine Learning



2 Supervised Learning

Supervised learning is a type of machine learning in which a model is trained using a labeled dataset, where each input data sample is associated with a known output. The objective of supervised learning is to learn a mapping function that accurately predicts the output for new, unseen input data.

2.1 Types of Supervised Learning

2.1.1 Classification

Classification is used when the output variable is categorical. Examples like Email spam detection, image recognition, disease classification.

2.1.2 Regression

Regression is used when the output variable is continuous or numerical.Examples are House price prediction, temperature forecasting.

3 Unsupervised Learning

Unsupervised learning is a type of machine learning in which the model is trained using unlabeled data, meaning that no predefined output or target variable is provided. The

primary objective of unsupervised learning is to discover hidden patterns, structures, or relationships within the data.

3.1 Types of Unsupervised Learning

3.1.1 Clustering

Clustering involves grouping data points such that samples within the same cluster are more similar to each other than to those in other clusters. Examples are Customer segmentation, document grouping, image segmentation.

3.1.2 Association

Association learning identifies interesting relationships or patterns among variables in large datasets. Examples are Market basket analysis, recommendation systems.

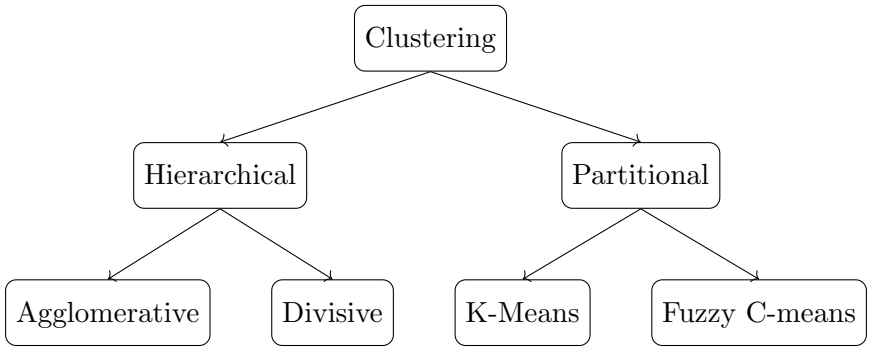
4 Reinforcement Learning

Reinforcement learning is a type of machine learning in which an agent learns to make decisions by interacting with an environment. The agent learns through a process of trial and error, receiving rewards or penalties for its actions, with the goal of maximizing the cumulative reward over time.

5 Algorithms

S.No	**Type of Machine Learning**	**Common Algorithms**
1	Supervised Learning (SL)	Linear Regression, Logistic Regression, Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbors (KNN)
2	Unsupervised Learning (USL)	K-Means Clustering, Hierarchical Clustering, DBSCAN, Principal Component Analysis (PCA)
3	Reinforcement Learning (RL)	Q-Learning, SARSA, Monte Carlo, Deep Q-Network (DQN)

6 Types of Clustering



6.1 Clustering Using DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that groups points based on spatial density. It automatically determines the number of clusters and identifies outliers as noise.

DBSCAN is particularly suitable for cone detection because:

- The number of cones is unknown in advance
- Cones form dense and compact point clusters
- Noise points can be safely ignored

7 Foundation of Linux Administration

7.1 Introduction to Linux and Shells

Linux is an open-source kernel that powers the majority of web servers and Android devices. A **Shell** is a command interpreter that provides a text-based interface to the operating system.

- **Bash:** (Bourne-again shell) The most common shell in Linux distributions
- **Zsh/Fish:** Popular alternatives with advanced auto-completion features
- **The man command:** Used to access the manual for any command (e.g `man ls`)

7.2 Navigation and File Management

Fundamental commands for interacting with the filesystem:

- `pwd`: Print Working Directory (shows where you are)

- `ls -al`: Lists all files, including hidden ones with detailed permissions
- `cd <path>`: Changes the current directory. Use `..` to move up one level
- `mkdir -p`: Creates nested directories.
- `touch`: Creates a new empty file or updates the timestamp of an existing one

7.3 File Manipulation and Search

- **Copy and Move:**

```
cp -r folder_source folder_destination  (Recursive copy)
mv old_name new_name  (Rename or move files)
```

- **Finding Files:** The `find` command is used for searching files based on criteria:

```
find . -name "*.js"   (Find all JS files)
find . -type f -mtime -1   (Files edited in last 24h)
```

- **Links:** `ln -s` creates symbolic links, which act like shortcuts to the original file

7.4 Permissions and Ownership

Security in Linux is managed through user/group ownership:

`chmod`: Change file modes/permissions (e.g `chmod 755 file.sh`)
`chown`: Change file owner and group
`sudo`: Execute a command with administrative (superuser) privileges

7.5 Processes and Environment

Monitoring system performance and managing background tasks:

- `ps` and `top`: Used to view currently running processes
- `kill <PID>`: Terminates a process using its Process ID
- `grep`: A powerful tool to search text within files or command output
- `xargs`: Converts standard input into arguments for another command

8 Introduction to ROS 2 Fundamentals and Development Workflow

ROS (Robot Operating System) is a framework used in many robotic applications to build and interconnect code. It provides a set of libraries and tools that allow developers to create software for robots, making it easier to build, test, and deploy new robotic systems.

8.1 ROS 2 CLI Tools

Category	Purpose	Example Commands
Configuring Environment	Set up ROS 2 workspace and environment variables	`source /opt/ros/humble /setup.bash`
Turtlesim, ros2 and rqt	Beginner-friendly simulation and visualization tools	`ros2 run turtlesim turtlesim_node`
Understanding Nodes	Manage and inspect active ROS 2 nodes	`ros2 node list`, `ros2 node info`
Understanding Topics	View and analyze message communication	`ros2 topic list`, `ros2 topic echo`
Launching Nodes	Start multiple nodes using launch files	`ros2 launch <package> <file>`

Table 1: Summary of ROS 2 CLI tools learned

8.2 Client Libraries and Build Tools

Tool	Function	Importance
Colcon	Build system for ROS 2 packages	Handles dependency resolution
Workspace	Organized directory for packages	Enables modular development
Python Client Library (rclpy)	Writing ROS 2 nodes in Python	Easy and beginner-friendly

Table 2: Client libraries and build tools

8.3 ROS 2 Workspace Structure

```
ros2_ws/
  src/
     my_package/
         my_package/
```

```
            publisher.py
            subscriber.py
        package.xml
        setup.py
build/
install/
log/
```

8.4 Development Workflow: Creating a Node

To build the Python Publisher and Subscriber, I followed the **Colcon Workflow**:

1. **Workspace Setup:** `mkdir -p  /ros2_ws/src`
2. **Package Creation:**

   ros2 pkg create –build-type ament_python –node-name my_node my_package

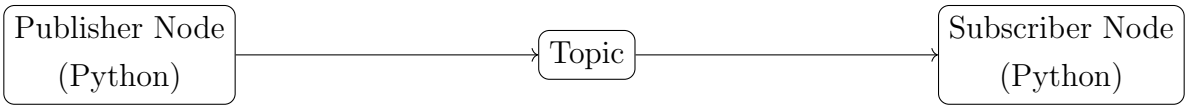
1. **Build:** `colcon build --symlink-install`
2. **Sourcing:** `source install/setup.bash`

8.5 Code Logic: Publisher/Subscriber Pattern

The core of the Python implementation relies on the `rclpy` library. Below is the simplified logic:

- **Publisher:** Inherits from `Node`, creates a timer, and calls `self.publisher_.publish(msg)`.
- **Subscriber:** Defines a `callback` function that executes every time data is received on the topic.

8.6 Publisher and Subscriber Communication Model



9 LiDAR-only Cone Detection Using ROS 2

9.1 Problem Statement

The objective of this project is to detect traffic cones using only LiDAR point cloud data recorded in a ROS 2 rosbag file. The task involves identifying individual cones present in the environment and computing the center position (x, y) of each detected cone without using camera or GPS data.

The input data is provided as a prerecorded rosbag file (`cones.db3`), which publishes LiDAR point cloud messages on a ROS 2 topic. The output of the system is the estimated center position of each cone printed in real time while the rosbag is played.

9.2 Sensor Used: LiDAR

LiDAR (Light Detection and Ranging) is a distance-measuring sensor that emits laser pulses and calculates distance based on the time taken for the reflected light to return. Each LiDAR scan produces a *point cloud*, which is a collection of points in three-dimensional space. Each point in the point cloud consists of:

- x — forward/backward position
- y — left/right position
- z — height from the ground
- intensity — strength of reflected laser signal

In ROS 2, LiDAR data is commonly published using the message type:
`sensor_msgs/msg/PointCloud`.

9.3 Point Cloud Representation

A point cloud is a large set of unordered 3D points representing the surrounding environment. Objects such as traffic cones appear as dense clusters of points, while flat surfaces such as roads form continuous planar regions.

9.4 Cone Detection Pipeline

The LiDAR-only cone detection system follows a structured perception pipeline consisting of four major stages:

1. **Point Cloud Acquisition:** The ROS 2 node subscribes to the LiDAR topic and receives `PointCloud` messages.

2. **Ground Removal:** Points belonging to the ground are removed by applying a height threshold on the z coordinate. Points with z values below the threshold are discarded.

3. **Clustering:** The remaining points are grouped using the DBSCAN clustering algorithm to identify individual cones.

4. **Cone Center Estimation:** For each detected cluster, the mean of the (x, y) coordinates is computed to estimate the cone center.

9.5 Use of Rosbag

A rosbag is a recorded dataset containing ROS topic messages. Instead of using a live LiDAR sensor, the provided rosbag file (`cones.db3`) is played back to simulate real-time sensor input.When the rosbag is played, all topics behave exactly as if a real sensor were publishing data. No cone positions are hardcoded, and all detections are computed dynamically from LiDAR data.

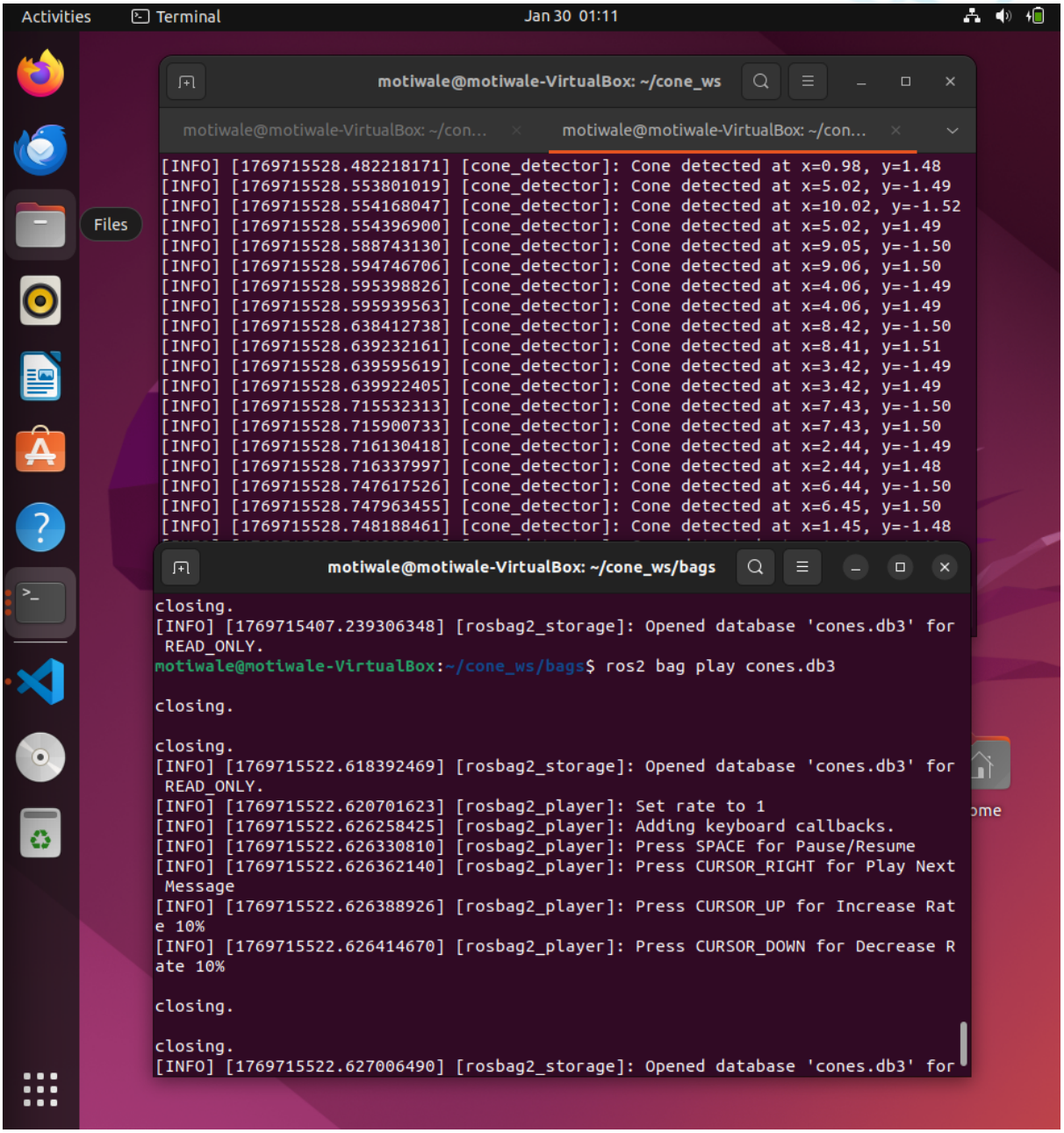
9.6 Final Output and Results

The final output of the LiDAR-only cone detection system consists of the estimated center positions of all detected traffic cones in the environment. For each cone, the system computes the mean of the clustered LiDAR points in the (x, y) plane and prints the result to the terminal in real time.

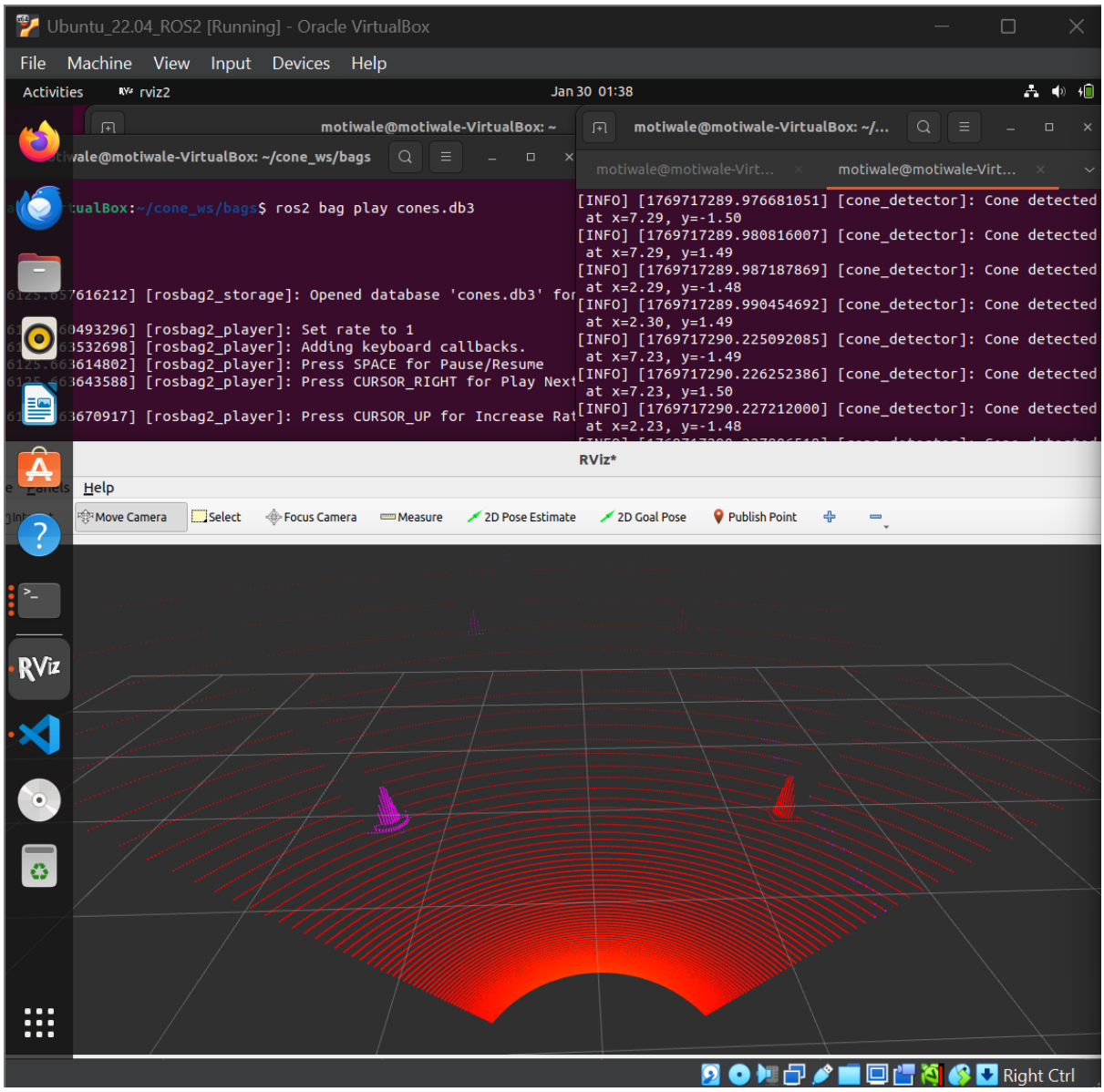
As the rosbag is played, the node continuously processes incoming point cloud data and reports cone positions dynamically. Each printed line corresponds to a distinct cone detected in the scene, confirming successful ground removal, clustering, and center estimation.

In addition to numerical output, the results were visually validated using RViz. The detected cone locations aligned well with the cone positions observed in the LiDAR point cloud visualization, providing confidence in the correctness and robustness of the implemented pipeline.

Overall, the final output demonstrates that accurate object-level perception can be achieved using only LiDAR data, without relying on cameras or prior knowledge of cone positions.


Activities
Terminal
Jan 30 01:11
Files
motiwale@motiwale-VirtualBox: ~/cone_ws
motiwale@motiwale-VirtualBox: ~/con...
motiwale@motiwale-VirtualBox: ~/con...
[INFO] [1769715528.482218171] [cone_detector]: Cone detected at x=0.98, y=1.48
[INFO] [1769715528.553801019] [cone_detector]: Cone detected at x=5.02, y=-1.49
[INFO] [1769715528.554168047] [cone_detector]: Cone detected at x=10.02, y=-1.52
[INFO] [1769715528.554396900] [cone_detector]: Cone detected at x=5.02, y=1.49
[INFO] [1769715528.588743130] [cone_detector]: Cone detected at x=9.05, y=-1.50
[INFO] [1769715528.594746706] [cone_detector]: Cone detected at x=9.06, y=1.50
[INFO] [1769715528.595398826] [cone_detector]: Cone detected at x=4.06, y=-1.49
[INFO] [1769715528.595939563] [cone_detector]: Cone detected at x=4.06, y=1.49
[INFO] [1769715528.638412738] [cone_detector]: Cone detected at x=8.42, y=-1.50
[INFO] [1769715528.639232161] [cone_detector]: Cone detected at x=8.41, y=1.51
[INFO] [1769715528.639595619] [cone_detector]: Cone detected at x=3.42, y=-1.49
[INFO] [1769715528.639922405] [cone_detector]: Cone detected at x=3.42, y=1.49
[INFO] [1769715528.715532313] [cone_detector]: Cone detected at x=7.43, y=-1.50
[INFO] [1769715528.715900733] [cone_detector]: Cone detected at x=7.43, y=1.50
[INFO] [1769715528.716130418] [cone_detector]: Cone detected at x=2.44, y=-1.49
[INFO] [1769715528.716337997] [cone_detector]: Cone detected at x=2.44, y=1.48
[INFO] [1769715528.747617526] [cone_detector]: Cone detected at x=6.44, y=-1.50
[INFO] [1769715528.747963455] [cone_detector]: Cone detected at x=6.45, y=1.50
[INFO] [1769715528.748188461] [cone_detector]: Cone detected at x=1.45, y=-1.48
motiwale@motiwale-VirtualBox: ~/cone_ws/bags
closing.
[INFO] [1769715407.239306348] [rosbag2_storage]: Opened database 'cones.db3' for READ_ONLY.
motiwale@motiwale-VirtualBox:~/cone_ws/bags$ ros2 bag play cones.db3
closing.
closing.
[INFO] [1769715522.618392469] [rosbag2_storage]: Opened database 'cones.db3' for READ_ONLY.
[INFO] [1769715522.620701623] [rosbag2_player]: Set rate to 1
[INFO] [1769715522.626258425] [rosbag2_player]: Adding keyboard callbacks.
[INFO] [1769715522.626330810] [rosbag2_player]: Press SPACE for Pause/Resume
[INFO] [1769715522.626362140] [rosbag2_player]: Press CURSOR_RIGHT for Play Next Message
[INFO] [1769715522.626388926] [rosbag2_player]: Press CURSOR_UP for Increase Rate 10%
[INFO] [1769715522.626414670] [rosbag2_player]: Press CURSOR_DOWN for Decrease Rate 10%
closing.
closing.
[INFO] [1769715522.627006490] [rosbag2_storage]: Opened database 'cones.db3' for

9.7 Visualization using rViz



RViz was used to visualize the LiDAR point cloud and validate the cone detection results. Visualization provided immediate spatial feedback, helping verify ground removal, clustering accuracy, and cone center estimation.

9.8 Learning Outcomes

Through the successful completion of this project, the following learning outcomes were achieved:

- Understanding of LiDAR sensor principles and point cloud data representation.
- Hands-on experience with ROS 2 architecture, including nodes, topics, subscribers, and rosbags.

- Practical implementation of a perception pipeline involving ground removal, clustering, and object detection.
- Application of the DBSCAN algorithm for real-world spatial clustering problems.
- Experience in processing high-dimensional sensor data using Python and NumPy.
- Use of RViz for visualization, debugging, and validation of perception results.
- Improved ability to design and evaluate modular robotic software systems.

The project demonstrates how meaningful object detection can be achieved using only LiDAR data in a ROS 2 environment.